

Bonded knots

Katarzyna Gozdek

26.05.2026

Abstract

The main objective of this paper is to present a systematic classification of uncolored bonded knots with singularity number at most nine. Bonded knots can model proteins that contain strong bonds between distant amino acids, such as ionic bonds or disulfide bridges. We describe the methods used to achieve this classification, including the generation of bonded knot diagrams and the distinction of topological knotted types through Yamada polynomial and Reidemeister moves.

1 Introduction

Approximately 1% of all structures inside Protein Data Bank constitute knotted proteins [1] - rare biomolecules whose backbones entangle themselves in a knot. Although the folding process leading to such topology is less efficient than that of their unknotted counterparts [2], the knotted domains remain evolutionary conserved [3]. This can be attributed to several functional advantages afforded by such conformation - increased number of intra-chain contacts in the vicinity of the knotted core provides higher thermal and mechanical stability, preventing unfolding-coupled proteolysis, while knotted motifs create a favorable environment for protein active sites [4][5]. Therefore most knotted proteins are enzymes with knotted domains binding substrates/cofactors. For instance, HiYibK a deeply 31 knotted protein from the SPOUT superfamily exhibits a total loss of binding affinity for S-adenosyl methionine (SAM) after experimentally opening up its knotting loop [5]. This also explains why knotted proteins account for roughly 25% of secondary active transporters in mammals, as the knotted domains are responsible for substrate binding or serve as hinge points that trigger conformational transitions [1].

Therefore, as shown in [1] entanglement can be used not only to classify proteins, but also to understand their structure-based functions. While classical knots are already being used to encode information about knotted proteins [6], bonded knots can be applied to model and study such structures on an even more intricate level. This approach captures not only the spatial knotting of the backbone but also incorporates the chemical intra-chain bonds, representing disulfide bridges, hydrogen bonds, and salt bridges. These additional constraints are represented by embedded arcs, which distinguish bonded knots from classical knots. More generally, collections of multiple chains connected by additional intra- or intermolecular bonds can be modeled as bonded links. Since protein backbones are not closed curves, a suitable closure must be introduced to enable the application of knot-theoretic methods. Notably, this approach extends beyond knotted proteins and applies to a broader class of biomolecules containing intra-chain or intermolecular bonds. For instance, proteins with disulfide bonds alone constitute roughly 30% of all eukaryotic proteins [7], highlighting the potential relevance of bonded knots in structural biology. As of now, bonded structures have been classified up to 7 singularities (number of crossings and vertices) [8]. In this paper, we extend this classification to 9 singularities and describe the methodology used to obtain these results.

2 Definitions

A *graph* is a collection of edges and vertices such that each edge has a pair of vertices as its endpoints. A *spatial graph* is a graph embedded in the 3-sphere S^3 [9].

A *bonded knot* is defined as a tuple (L, b) , where:

- L is a knot embedded in the 3-sphere S^3 (or R^3),
- $b = \{b_1, b_2, \dots, b_n\}$ is a collection of pairwise disjoint bonds (closed intervals) properly embedded in $S^3 \setminus L$.

Bonded knots can be represented as 3-valent graphs equipped with a perfect matching - that is, there is a subset of edges that covers every vertex in the graph without any two edges sharing a mutual vertex [10, 11, 8].

A *bonded link* is defined analogously, where L is a link. The bonds may connect different components of the link or the same component.

2.1 Projections, Diagram Equivalence and Reidemeister Moves

A *knot diagram* is a regular projection of a knot onto a plane, with crossings equipped with over-under information [10].

The *Planar Diagram (PD) code* of a bonded knot is a combinatorial encoding of a diagram obtained by labeling its arcs and recording, for each node (crossing or bond endpoint), the incident arcs in counterclockwise order. In the case of a crossing, the ordering is initiated at an undercrossing arc. The resulting PD code is an ordered list of such node entries that completely determines the planar diagram [8].

Reidemeister moves are local transformations of knot diagrams that preserve knot type. There are three classical Reidemeister moves (I–III), together with additional moves IV and V for bonded knots (or knotted graphs), which describe interactions at vertices where bonds meet the knot.

Two knot diagrams are *equivalent* if there exists a finite sequence of Reidemeister moves transforming one diagram into the other [8].

The *singularity number* is defined as

$$s(K) = c(K) + 2b(K),$$

where $c(K)$ denotes the minimal number of crossings among all diagrams of a bonded knot K under generalized Reidemeister moves, and $b(K)$ is the number of bonds in K [8].

2.2 Diagram Terminology

A *vertex* is a bond endpoint. A vertex is represented in PD-code as $V[a_1, a_2, a_3]$, where the entries denote the incident arcs listed in counterclockwise order.

A *crossing* is a point where one strand passes over another. In the PD-code representation, the incident arcs are ordered counterclockwise, starting at an undercrossing arc; undercrossing arcs occupy odd positions and overcrossing arcs even positions. A crossing is denoted by $X[a_1, a_2, a_3, a_4]$ [10].

A *node* is either a vertex or a crossing.

An *arc* is a smooth curve joining a pair of adjacent nodes, with no vertices or crossings in its interior.

A *strand* is a simple path in the diagram whose endpoints are nodes, and whose interior contains no vertices. Crossings may occur along a strand.

An *edge* is a strand between two vertices.

A *diagram reduction* occurs when a diagram with n nodes can be transformed, via Reidemeister moves, into an equivalent diagram containing strictly fewer than n nodes.

METHODOLOGY

3 Classification method

How Am I to Know

The classification method used in this thesis largely follows the scheme presented in [8]. It involves generating all possible diagrams with vertices of degree 3 and 4, converting 4-valent vertices into crossings of both types (over- and under-crossings), computing invariants using the Yamada polynomial, and finally simplifying diagrams belonging to the same Yamada equivalence group in order to obtain unique diagrams.

1. Diagram generation:

All possible non-isomorphic graph diagrams with, respectively, 8, 9 and 10 vertices were generated produced using the plantri software package [12]. These were restricted with connectivity ≥ 1 , a minimum vertex degree of 1, and an edge count bounded by $\lceil n/2 \rceil \leq e \leq 2n$, where n is the number of vertices. These were further modified through edge parallelization to obtain structures containing only 3- and 4-valent vertices - additional parallel edges were added to vertices with degree lower than 4 until further additions would result in creating vertex with degree higher than maximum threshold 4. Since 4-valent vertices represent crossings and 3-valent vertices bond endpoints the total number of 3-valent vertices must be even. Taking this into account yielded us ? candidate graphs with singularity number 8, ? diagrams with singularity number 9 and ? diagrams with singularity number 10.

2. Over/under conversion:

Every 4-valent vertex is converted to a crossing of over and under type resulting in respectively ?, ? and ? diagrams for graphs with singularity number 8, 9 and 10.

3. Yamada polynomial grouping:

Systematic tools used for distinguishing classical knots consist mainly of knot invariants such as the Jones, Alexander, or Kauffman polynomials. However, these are generally not applicable to bonded knots. Moreover, the highly computationally efficient geometric tools utilized in classical knot tabulation are also unavailable, as they rely on the hyperbolic structures of knot complements, and bonded knots, in contrast to classical knots, mostly do not admit such structures.

Although newer invariants for bonded knots have recently been introduced [11, 13], the Yamada polynomial remains the most computationally efficient topological-vertex invariant for uncolored spatial graphs. The Yamada polynomial is an incomplete invariant, meaning it is possible for two non-isomorphic spatial graphs to yield identical polynomials. While spatial graphs that are distinctly grouped by their Yamada polynomials are not equivalent to each other, the graphs within the same polynomial group must still be tested for equivalence, usually through brute-force searches utilizing generalized Reidemeister moves. This makes the task of classifying bonded knots significantly more difficult than classical knot tabulation. This produces ? diagrams partitioned into ? groups whose equivalence has to be established.

4. Automated simplification:

The simplification algorithm presented in [8] uses sequences of Reidemeister moves to explore sets of diagrams reachable from the input states and performs a brute-force comparison, identifying two diagrams as equivalent whenever these sets intersect.

The search is limited by a prescribed crossing-increasing depth in the Reidemeister move space. Increasing this depth allows more equivalent diagrams to be identified, but also makes the search more computationally expensive.

The number of diagrams to be considered grows rapidly with the number of singularities. Therefore, in order to classify bonded knots with more than seven singularities, we modified the simplification approach to improve its efficiency and therefore allow us to classify diagrams with higher singularity numbers (see Section 4).

If, during this simplification process, a diagram is reduced to a structure with lower singularity number than currently under consideration, it is excluded from the final classification, because it has already been accounted for in the lower-order classifications.

5. Manual analysis:

This step is performed only for diagrams inside Yamada groups that were not classified as equivalent in the previous step.

These diagrams are handled using the following methods:

- **Distinct structural composition:** For example, a differing number of components. A single-component knot is immediately non-equivalent to a multi-component link.

- **Bond removal:** If removing the bond connecting the vertices transforms diagrams into fundamentally different structures - for example, one into a Hopf link and the other into a trefoil knot with an unknot – the original diagrams are not equivalent.
- **Mirror-image reduction:** Retaining only a single representative from each mirror pair in our classification allows us to quickly process groups containing mirror images without needing to establish their equivalence.

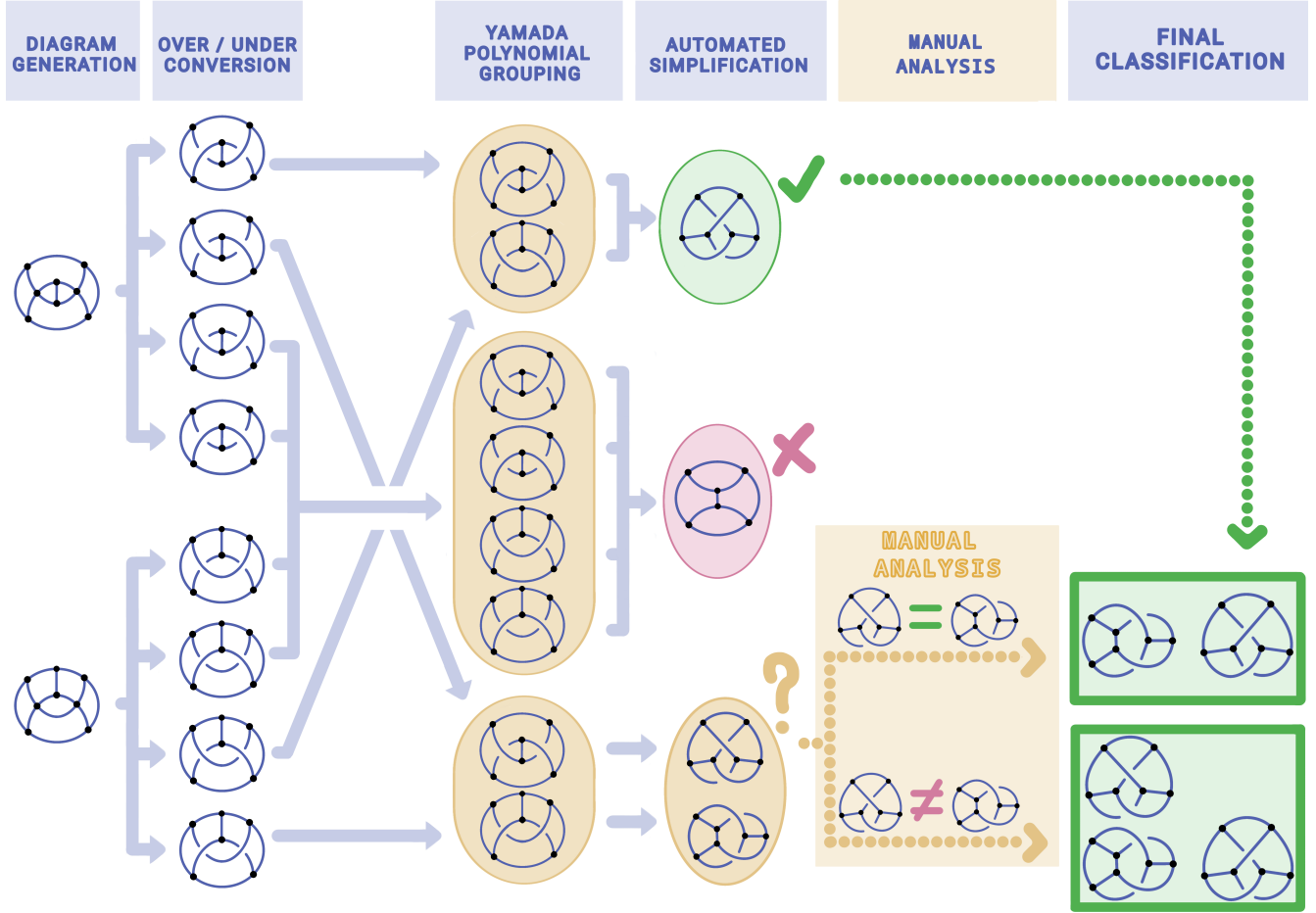


Figure 1: Overview of the proposed methodology and classification workflow.

Diagram generation, Yamada polynomial grouping, and automated simplification are marked in violet as steps that are always included in the workflow, while manual analysis is marked in yellow as a step that is conditional. In the 'Automated simplification' stage, three possible outcomes are shown:

- **Green:** All diagrams within a group are transformed to the same conformation, establishing them as equivalent, and allowing their immediate inclusion in the final classification.
- **Red:** A diagram is reduced to a structure with a fewer number of singularities than currently considered and therefore is not included in the final classification.
- **Yellow:** Automated simplification didn't converge on a common conformation during transformations and consequently diagrams equivalence could not be determined. This triggers the 'Manual analysis' stage, yielding two possible outcomes: diagrams are established as equivalent (denoted by $=$) and a single representative is added to the final classification, or they are proven to be distinct (denoted by $\neq$) and therefore are both added to the final classification.

*For the purpose of keeping the illustration as informative and accessible as possible, certain minor simplifications are present: shown reduced diagrams could, in practice, be discarded at an earlier stage and the diagrams in the yellow group whose equivalence remains undecided in the 'Automated simplification' step are actually confirmed as equivalent at this stage

4 Simplification Idea

Come Together

Instead of applying many different sequences of Reidemeister moves, we attempt to transform diagrams toward their outerplanar forms. This approach should allow equivalent diagrams to be identified more efficiently, since each move brings the diagrams closer to a common canonical form, rather than exhaustively exploring the Reidemeister move space.

Naturally, a diagram may have multiple outerplanar forms, particularly because our method allows the number of crossings to increase (see Section 2). Therefore, it is still necessary to explore the Reidemeister move space; however, when the diagrams are equivalent, the explored space is much smaller than in the original approach.

Simplification Algorithm Overview

The input diagrams whose equivalence is to be determined are initially added to a queue. During each iteration, one diagram is removed from the queue and processed as follows:

- Its canonical form is computed and stored together with the input diagram from which it was obtained.
- If the same canonical form has previously been obtained from a different input diagram, the corresponding diagrams are identified as equivalent.
- Equivalence is treated as a transitive relation. Therefore, if diagrams A and B are equivalent, and diagrams C and D are equivalent, then discovering that B and C are equivalent merges the two equivalence classes into a single class containing A, B, C, and D.
- Diagrams already identified as outerplanar are discarded from further exploration.
- To prevent cycles, each visited state is stored as a pair consisting of its canonical form and the length of its outer face, since the same canonical form may occur with different outer-face configurations.
- Diagrams that have already been visited are not explored again.
- Successor states are generated by applying transformations intended to guide the diagram toward an outerplanar form. During this process, all possible outer faces of the recorded length are considered.
- The generated diagrams are added to the queue.

The algorithm terminates when all diagrams belong to a single equivalence class, the search space is exhausted, or a predefined time limit is reached.

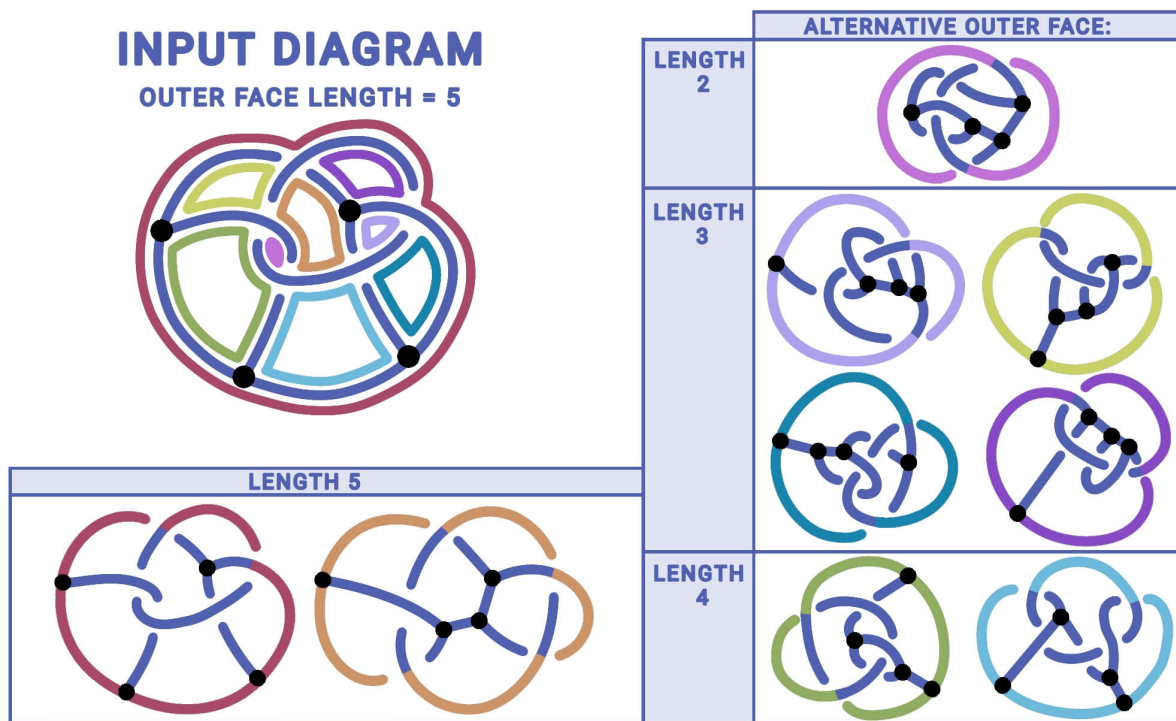


Figure 2: Dla każdego digarmu można wybrać outer-face na wiele sposobów. Zaznaczone kolorami 'pętle' na obrazku 'input diagram' zawsze mogą znaleźć się na zewnątrz nie zmieniając przy tym pd-codu. Narysowałam ten sam diagram, ale z wszystkimi możliwymi fragmentami na zewnątrz - zewnętrzny 'outer face' jest zaznaczony kolorem odpowiadającym danej pętli w 'input diagram'. Długość outer-face to ilość segmentów między kolejnymi wierzchołkami (V/X) na outer-face.

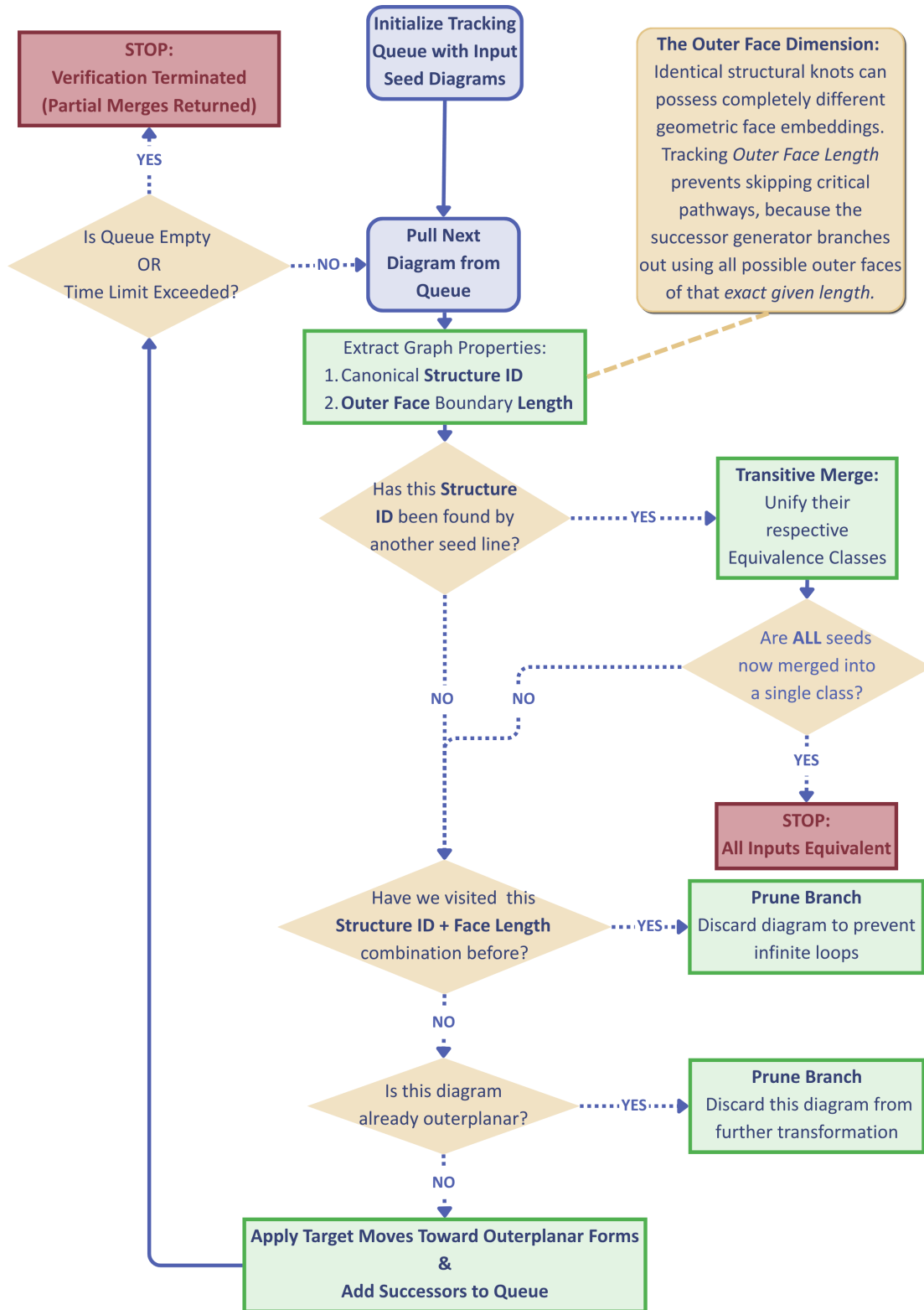


Figure 3: The visual workflow of the simplification algorithm.

4.1 Successor State Generation

I Want to Break Free (to the outer face)

Once the outer face has been fixed, the vertices lying in the interior of the diagram can be identified. The outer face consists of segments, where each segment is bounded by two consecutive outer-face nodes.

For the initial diagram, the following steps are performed for each segment to produce a set of alternative transformed diagrams:

1. Identify the inner vertex closest to the selected outer-face segment.
2. Move this vertex toward the outer face through the selected outer-face segment.
3. Reduce diagram: the transformation from step 2 often produces a diagram that can be simplified using the reduction procedure described in Section 3. The resulting diagram is therefore simplified repeatedly until no further reduction moves are applicable. Reduction moves that would return a newly introduced outer-face vertex to the interior of the diagram are not performed.

Consequently, this parallel operation transforms the initial diagram into a set of distinct diagrams, each corresponding to an expansion through a different outer-face segment. Although the transformation step aims to move additional vertices onto the outer face, subsequent reduction operations may alter the structure of the diagram in such a way that some vertices previously lying on the outer face become interior vertices again.

Cycles may occur in the search process, which is why previously visited states are stored, as described in Section 2.

The proposed procedure does not guarantee that an outerplanar form will always be reached. Nevertheless, equivalent diagrams may still be identified because the search process can lead to common intermediate states even when an outerplanar representation is not obtained.

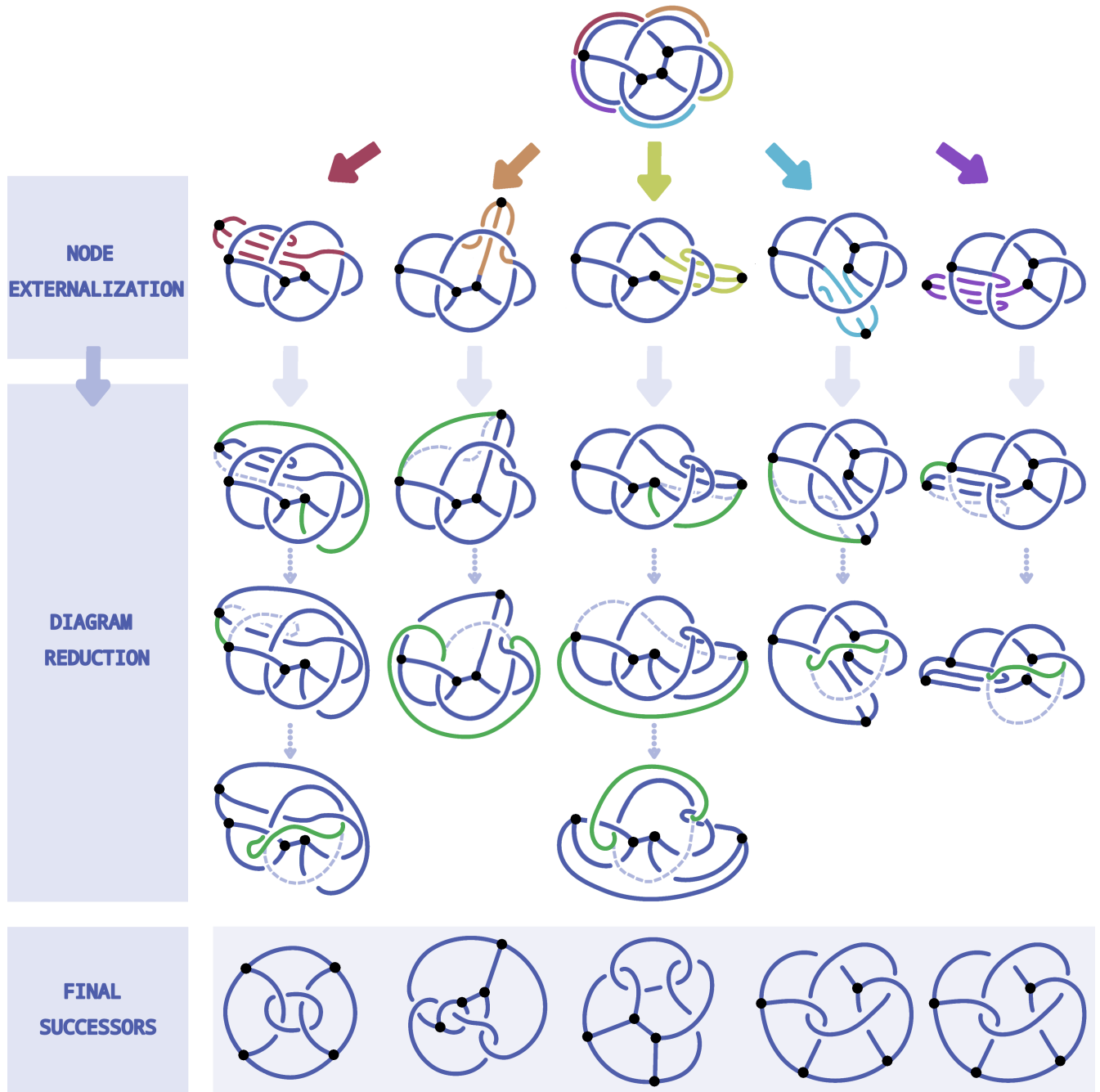


Figure 4: Dla danego diagramu z określonym outer-face tworzymy 'następniki' wyciągając przez wszystkie segmenty najbliższy im wierzchołek, tzn. dla diagramu z outer-face długości 5 dostajemy 5 nowych diagramów. Final successors to po prostu diagram z diagram eduction narysowany ładniej.

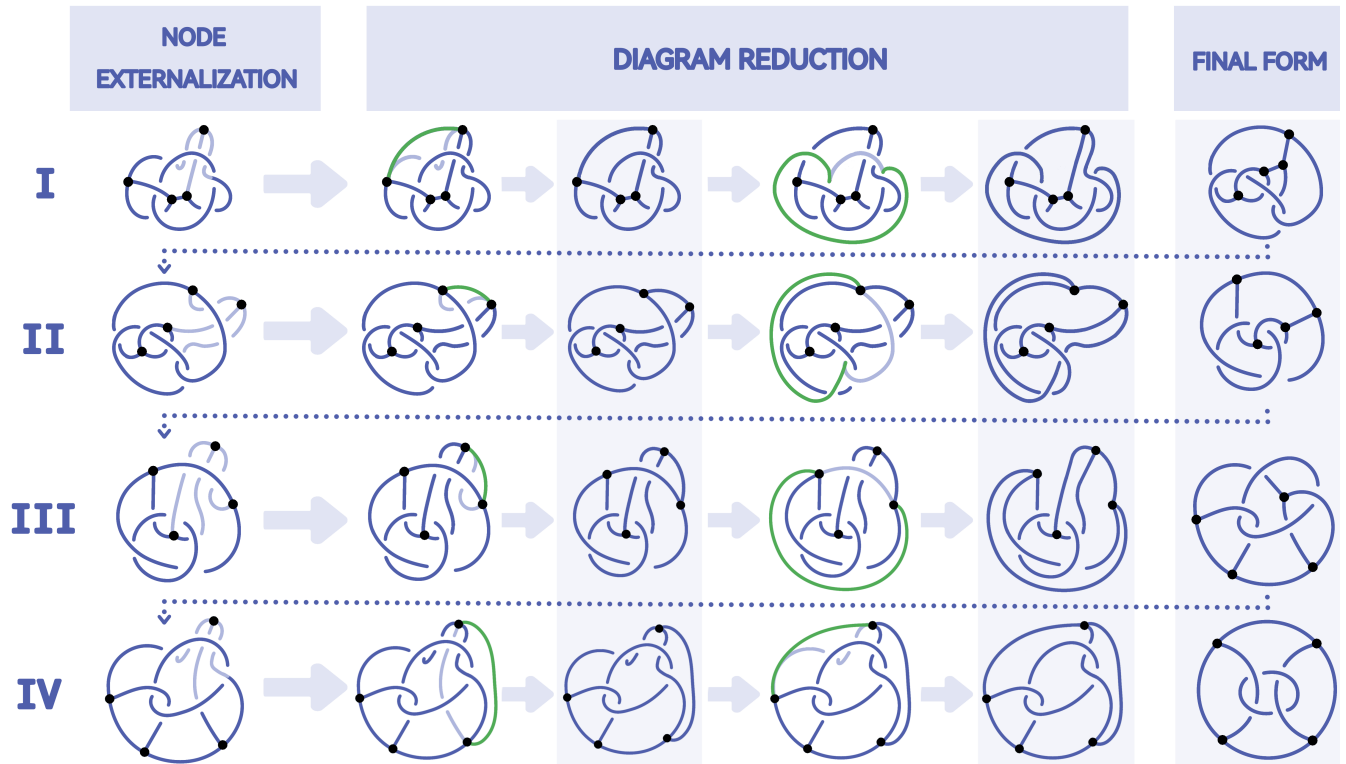


Figure 5: Ilustracja 4 iteracji algorytmu. Wizualizacja uproszczona tzn. podążamy jedną ścieżką w przestrzeni stanów (na rysunku jest wybrany 1 outer face oraz wyciągany wierzchołek na 1 sposób, a na pełnej ilustracji byłoby rozgałęzienie przed 'node externalization' na wszystkie outer-face danej długości, a potem w 'node externalization' kolejne rozgałęzienie na wszystkie możliwe sposoby wyciągnięcia wierzchołka.) Jak widać naszym algorytmem dochodzimy (zwykle) do postaci outerplanarnej. Są też pętle - final form z 3 iteracji to tak naprawdę input diagram z innym outer-face (ścieżka nie została przerwana ze względu na inny outer-face)

4.1.1 Identifying the inner vertex closest to the outer face.

The interior vertices and the nodes lying on the outer face are first identified. These are then used as starting and ending points for the minimal-crossing path-finding function. The minimal-crossing path function (see Section ...) determines a path between a given pair of nodes that minimizes the number of crossings introduced by inserting a strand along the path.

For each target outer-face segment (i.e., the strand between two adjacent vertices), one path is determined. This procedure identifies not only the interior vertex closest to each outer-face segment, but also the corresponding paths along which the vertices can be moved onto the outer face.

[Obrazek]

4.1.2 Moving the Inner Vertex onto the Outer Face

Multiple paths are identified in the previous step, and a new diagram is generated for each of them as follows.

Since each vertex has three incident edges, this procedure requires inserting three new strands along the path associated with the selected interior vertex (see previous section). These strands are then connected outside the

diagram to form a new outer-face vertex. Finally, the remaining ends of the inserted strands are connected to the corresponding endpoints of the original vertex, thereby integrating the new structure into the diagram.

Moving a vertex onto the outer face introduces new crossings. Whether these are assigned as overcrossings or undercrossings is determined by the immediate neighborhood of the vertex:

- If the majority of the three incident strands terminate at overcrossings, the newly introduced crossings are assigned as overcrossings.
- Otherwise, the newly introduced crossings are assigned as undercrossings.

This heuristic increases the likelihood that the reduction step (see Section ...) will remove some of the newly introduced crossings, thereby reducing the number of crossings retained in the final diagram.

[Obrazek]

4.1.3 Diagram-Reducing Strand Flip Move

Idea

When a strand crosses exclusively over all other strands it encounters in the diagram (Fig. 1), it is possible to perform a *strand flip* - move the strand to an alternative position in the diagram in such a way that all of its current crossings disappear. This operation may introduce new crossings over other strands, depending on the chosen placement (Fig. 2a). Consequently, if the alternative position induces fewer crossings than the original, then the total number of crossings in a diagram decreases and the diagram is therefore reduced (Fig. 2b, 2c).

The idea above is described for strands that pass exclusively over other strands they encounter. By symmetry, the same reasoning holds for strands that pass exclusively under all encountered crossings.

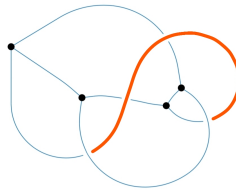
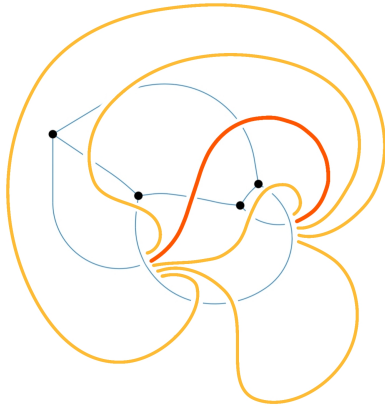
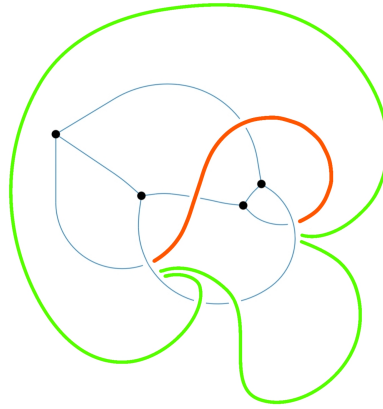


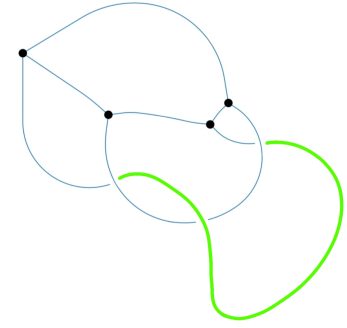
Figure 6: over-over path example



(a) Alternative path placement example



(b) better paths example



(c) Diagram after reduction

Figure 7: Caption

Algorithm: Detection of Diagram-Reducing Strand Flip Move

For simplicity, the following algorithm description focuses solely on strands that pass exclusively over crossings and is analogous to the case of strands passing exclusively under crossings.

For every pair of nodes in a diagram (vertices and crossings):

1. If paths passing exclusively over crossings exist, find the one containing the largest number of crossings.
2. Compute all possible paths between the nodes and find the one that contains the smallest number of crossings.
3. If the number of crossings in the minimal-crossing path (2) is strictly smaller than in the maximal over-type path (1), a diagram reducing strand flip move is possible.

Finding an Over-Type Path

Finding an 'over'-type path between two nodes is performed by following each strand incident to the starting node and tracing it through the diagram. The process terminates either unsuccessfully, if a vertex or an 'under' crossing is encountered, or successfully if the target node is reached. In the latter case, the traced path is the desired 'over'-type path.

Finding a Minimal-Crossing Path

Computing an alternative placement for a strand, together with its cost, which represents the number of crossings that would be created by inserting a strand along this path, can be obtained by:

1. Following an arc until a crossing or a vertex is reached
2. At a vertex, the two remaining arcs must be considered. These correspond to continuing the traversal with either a left or a right turn.
3. At a crossing, three arcs may be followed: one to the left, one to the right and one passing straight through the crossing.

The side of the path along which the strand is inserted determines whether a crossing is created. Taking this into account, the following cost model is defined:

- Turns at vertices or crossings have cost 1 if the side changes and cost 0 otherwise.
- Passing straight through a crossing preserves the side and has cost 1.

The minimal-crossing path is then obtained by selecting the path with the minimal total cost according to this cost model.

5 Results

All merged through pipeline2:

33 yamada groups

Some diagrams were reduced to less than 8 singularities
Only 8

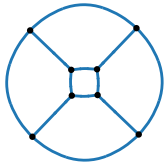
Table 1: Only 8: Bonded knots and their Yamada polynomials

Name	Chirality	Yamada
$B(0, 4)_1$		$-A^{10} + 2A^9 - 7A^8 + 5A^7 - 14A^6 + 6A^5 - 14A^4 + 5A^3 - 7A^2 + 2A - 1$
$B(2, 3)_1$		$-A^{11} - A^{10} - A^9 - 4A^8 + A^7 - 5A^6 + 3A^5 - 3A^4 + 3A^3 + A + 1$
$B(2, 3)_2$		$A^{12} - A^{11} + 2A^{10} + 2A^9 + A^8 + 6A^7 + 6A^5 + 3A^3 + 2A^2 + 2$
$B(2, 3)_3$		$-A^{11} - A^{10} - 3A^8 + 3A^7 - 3A^6 + 5A^5 - A^4 + 4A^3 + A^2 + A + 1$
$B(2, 3)_4$		$A^{12} - A^{11} - 4A^8 + A^7 - 6A^6 + A^5 - 4A^4 - A + 1$
$B(2, 3)_5$		$2A^{12} + 2A^{10} + 3A^9 + 6A^7 + 6A^5 + A^4 + 2A^3 + 2A^2 - A + 1$
$B(4, 2)_1$		$-A^{16} + A^{14} - A^{13} + 2A^{12} + A^{11} - A^{10} + A^9 - 3A^8 - 3A^6 - A^5 - A^3 + A - 1$
$B(4, 2)_2$		$-A^{16} - A^{13} + A^{12} + A^{11} + 3A^9 + 3A^7 + A^5 + A^4 - A^3 - 1$
$B(4, 2)_3$		$A^{16} - A^{15} + 2A^{14} + A^{13} + 2A^{11} - 2A^{10} + A^9 - 2A^8 + A^7 + A^6 + A^5 + 2A^4 + 2A^3 + A^2 + A + 1$
$B(4, 2)_4$		$A^{15} + 2A^{12} + A^{11} + A^{10} + 3A^9 + 3A^7 + 2A^5 + A^4 - A^3 + A^2 - A - 1$
$B(4, 2)_5$		$A^{16} + 2A^{14} + A^{13} + 2A^{11} - A^{10} + 2A^9 - A^8 + A^7 + A^4 + A^3 + A^2 + A + 1$
$B(4, 2)_6$		$-A^{16} - A^{14} - 2A^{13} - A^{11} - 2A^{10} - 3A^8 - 2A^6 + A^4 - A^3 + A^2 - 1$
$B(4, 2)_7$		$-A^{16} + A^{15} + A^{14} - A^{13} + 2A^{12} + A^{11} + 3A^9 + 3A^7 + A^5 + 2A^4 - A^3 + A^2 + A - 1$
$B(4, 2)_8$		$A^{15} + A^{14} - A^{13} + A^{12} - A^{11} - 2A^{10} - 3A^8 - 3A^6 - A^5 - A^4 - 2A^3 - 1$
$B(4, 2)_9$		$-A^{16} + A^{14} - A^{13} + A^{12} - 2A^{10} - 3A^8 - 2A^6 - A^5 - 2A^3 - A^2 - 1$
$B(4, 2)_{10}$		$-A^{16} + A^{15} - A^{13} - A^{11} - 3A^{10} - 3A^8 + A^7 - A^6 + A^5 + 2A^4 - A^3 + A^2 - 1$
$B(4, 2)_{11}$		$-2A^{16} - 2A^{13} + A^{12} + A^{11} - A^{10} + 2A^9 - 2A^8 + A^7 - 2A^6 - A^5 + A^4 - 2A^3 + A - 1$
$B(4, 2)_{12}$		$A^{16} + A^{15} + A^{14} + A^{13} + A^{12} + A^9 - A^8 + 2A^7 - A^6 + 2A^5 + A^3 + 2A^2 + 1$
$B(4, 2)_{13}$		$-A^{16} + A^{15} - 2A^{13} + A^{12} - A^{11} - 2A^{10} + A^9 - 2A^8 + 2A^7 - A^6 + A^5 + A^4 - 2A^3 - 2$
$B(4, 2)_{14}$		$A^{16} + A^{15} + A^{14} + 2A^{13} + 2A^{12} + A^{11} + A^{10} + A^9 - 2A^8 + A^7 - 2A^6 + 2A^5 + A^3 + 2A^2 - A + 1$
$B(4, 2)_{15}$		$-A^{14} - A^{12} - A^{11} - A^{10} - 2A^8 + A^7 - 2A^6 + A^5 - A^4 + A^2 - A + 1$
$H(4, 2)_1$		$-2A^{15} - 2A^{13} - 2A^{12} - 2A^{10} + A^9 - A^8 + A^7 + A^6 + 2A^4 + A^3 + A^2 + A + 1$
$H(4, 2)_2$		$A^{15} + A^{14} + 2A^{12} + A^9 - A^8 + A^7 - A^6 - 2A^3 - A - 1$
$H(4, 2)_3$		$-A^{15} - A^{14} - A^{13} - A^{12} - 2A^{11} - A^9 - A^8 + A^7 - A^6 + 2A^5 + 2A^3 + 2A^2 + 2$
$H(4, 2)_4$		$A^{16} - 2A^{11} - A^8 + 2A^7 - A^6 + 2A^5 - A^4 + A^2 - 2A + 1$
$H(4, 2)_5$		$A^{16} - 2A^{15} + A^{14} - A^{12} + 2A^{11} - A^{10} + 2A^9 - A^8 - 2A^5 + 1$

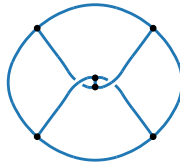
Table 2: Only 8: PD codes of bonded knots

Name	PD Codes
$B(0, 4)_1$	$V[0, 1, 2], V[0, 3, 4], V[1, 5, 6], V[2, 7, 8], V[3, 8, 9], V[4, 10, 5], V[6, 11, 7], V[9, 11, 10]$
$B(2, 3)_1$	$V[0, 1, 2], V[0, 3, 4], V[1, 5, 6], V[7, 8, 9], V[10, 9, 11], V[4, 12, 5], X[11, 8, 6, 12], X[3, 2, 7, 10]$
$B(2, 3)_2$	$V[0, 1, 2], V[3, 4, 5], V[6, 7, 4], V[2, 8, 9], V[0, 10, 3], V[10, 9, 6], X[1, 5, 11, 12], X[7, 8, 12, 11]$
$B(2, 3)_3$	$V[0, 1, 2], V[0, 3, 4], V[1, 5, 6], V[7, 8, 9], V[10, 11, 8], V[4, 7, 5], X[9, 11, 12, 6], X[2, 12, 10, 3]$
$B(2, 3)_4$	$V[0, 1, 2], V[3, 4, 5], V[6, 7, 8], V[9, 0, 6], V[10, 5, 1], V[9, 11, 10], X[4, 12, 7, 2], X[11, 8, 12, 3]$
$B(2, 3)_5$	$V[0, 1, 2], V[3, 4, 5], V[6, 7, 4], V[8, 2, 9], V[10, 3, 0], V[10, 8, 6], X[5, 11, 12, 1], X[9, 12, 11, 7]$
$B(4, 2)_1$	$V[0, 1, 2], V[0, 3, 4], V[2, 5, 6], V[3, 6, 7], X[1, 4, 8, 9], X[10, 11, 12, 8], X[7, 13, 11, 10], X[9, 12, 13, 5]$
$B(4, 2)_2$	$V[0, 1, 2], V[0, 3, 4], V[3, 5, 6], V[6, 7, 8], X[9, 10, 11, 2], X[8, 12, 10, 13], X[12, 7, 5, 11], X[1, 4, 13, 9]$
$B(4, 2)_3$	$V[0, 1, 2], V[3, 0, 4], V[5, 6, 1], V[3, 7, 5], X[8, 9, 10, 2], X[11, 12, 9, 8], X[4, 10, 13, 7], X[12, 11, 6, 13]$
$B(4, 2)_4$	$V[0, 1, 2], V[0, 3, 4], V[2, 5, 6], V[3, 7, 8], X[4, 9, 10, 1], X[11, 12, 9, 8], X[12, 13, 5, 10], X[7, 6, 13, 11]$
$B(4, 2)_5$	$V[0, 1, 2], V[0, 3, 4], V[1, 5, 6], V[4, 7, 5], X[3, 2, 8, 9], X[10, 11, 9, 8], X[12, 13, 11, 10], X[13, 12, 6, 7]$
$B(4, 2)_6$	$V[0, 1, 2], V[0, 3, 4], V[5, 6, 7], V[8, 7, 9], X[10, 5, 1, 4], X[8, 11, 12, 2], X[3, 12, 11, 13], X[13, 9, 6, 10]$
$B(4, 2)_7$	$V[0, 1, 2], V[3, 4, 5], V[6, 0, 7], V[6, 3, 8], X[9, 10, 11, 2], X[5, 9, 12, 13], X[13, 12, 1, 8], X[7, 11, 10, 4]$
$B(4, 2)_8$	$V[0, 1, 2], V[0, 3, 4], V[3, 5, 6], V[7, 8, 9], X[2, 10, 11, 5], X[9, 12, 10, 1], X[13, 11, 12, 8], X[6, 13, 7, 4]$
$B(4, 2)_9$	$V[0, 1, 2], V[0, 3, 4], V[1, 5, 6], V[4, 7, 5], X[8, 9, 10, 11], X[12, 13, 9, 6], X[7, 10, 13, 12], X[2, 8, 11, 3]$
$B(4, 2)_{10}$	$V[0, 1, 2], V[3, 4, 5], V[5, 6, 1], V[3, 0, 7], X[8, 9, 10, 11], X[7, 12, 11, 4], X[6, 10, 9, 13], X[13, 8, 12, 2]$
$B(4, 2)_{11}$	$V[0, 1, 2], V[3, 4, 5], V[0, 3, 6], V[6, 7, 8], X[2, 9, 10, 4], X[11, 10, 9, 12], X[8, 13, 12, 1], X[5, 11, 13, 7]$
$B(4, 2)_{12}$	$V[0, 1, 2], V[0, 3, 4], V[1, 5, 6], V[4, 7, 5], X[2, 8, 9, 3], X[10, 11, 12, 13], X[8, 12, 11, 9], X[7, 10, 13, 6]$
$B(4, 2)_{13}$	$V[0, 1, 2], V[3, 4, 5], V[0, 3, 6], V[6, 7, 8], X[9, 10, 4, 2], X[11, 12, 10, 9], X[1, 8, 13, 11], X[12, 13, 7, 5]$
$B(4, 2)_{14}$	$V[0, 1, 2], V[3, 0, 4], V[5, 6, 1], V[3, 7, 5], X[2, 8, 9, 10], X[8, 11, 12, 9], X[7, 4, 10, 13], X[13, 12, 11, 6]$
$B(4, 2)_{15}$	$V[0, 1, 2], V[3, 4, 5], V[0, 6, 7], V[7, 8, 9], X[6, 2, 3, 10], X[10, 11, 12, 8], X[9, 13, 4, 1], X[13, 12, 11, 5]$
$H(4, 2)_1$	$V[0, 1, 2], V[0, 3, 4], V[5, 6, 7], V[3, 5, 8], X[2, 9, 10, 6], X[11, 12, 7, 10], X[12, 11, 13, 8], X[4, 13, 9, 1]$
$H(4, 2)_2$	$V[0, 1, 2], V[0, 3, 4], V[1, 5, 6], V[7, 8, 9], X[6, 9, 10, 11], X[12, 10, 8, 13], X[13, 7, 5, 4], X[11, 12, 3, 2]$
$H(4, 2)_3$	$V[0, 1, 2], V[3, 4, 1], V[5, 6, 7], V[5, 8, 9], X[6, 2, 4, 10], X[10, 11, 12, 7], X[13, 3, 0, 9], X[8, 12, 11, 13]$
$H(4, 2)_4$	$V[0, 1, 2], V[3, 4, 5], V[6, 7, 8], V[6, 3, 0], X[9, 10, 1, 5], X[11, 9, 4, 8], X[10, 12, 13, 2], X[7, 13, 12, 11]$
$H(4, 2)_5$	$V[0, 1, 2], V[3, 4, 5], V[6, 7, 8], V[6, 3, 0], X[5, 9, 10, 1], X[8, 11, 9, 4], X[2, 10, 12, 13], X[13, 12, 11, 7]$

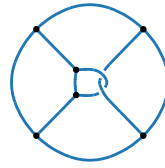
Knot Diagrams - 8 singularities



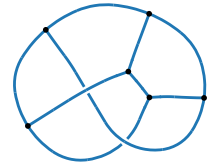
$B(0, 4)_1$



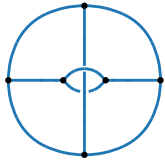
$B(2, 3)_1$



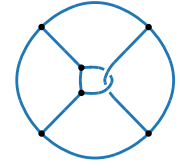
$B(2, 3)_2$



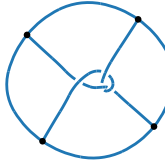
$B(2, 3)_3$



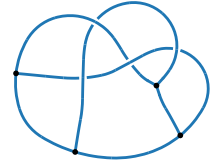
$B(2, 3)_4$



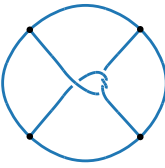
$B(2, 3)_5$



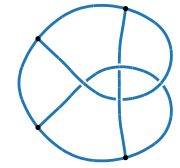
$B(4, 2)_1$



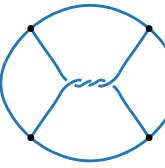
$B(4, 2)_2$



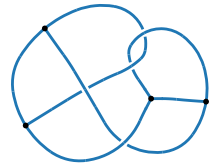
$B(4, 2)_3$



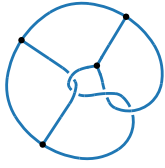
$B(4, 2)_4$



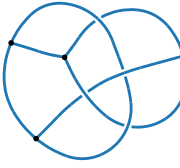
$B(4, 2)_5$



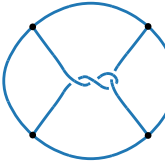
$B(4, 2)_6$



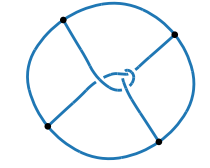
$B(4, 2)_7$



$B(4, 2)_8$



$B(4, 2)_9$



$B(4, 2)_{10}$

Figure 8: Bonded knots. Each entry is represented by two rows: the diagram and its label.

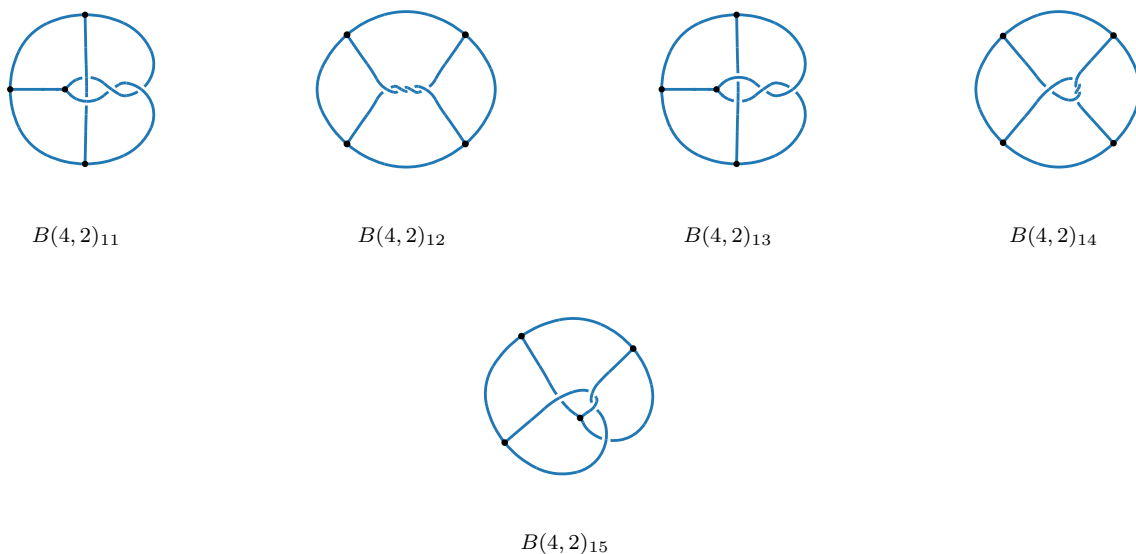


Figure 9: Bonded knots (continued). Each entry is represented by two rows: the diagram and its label.

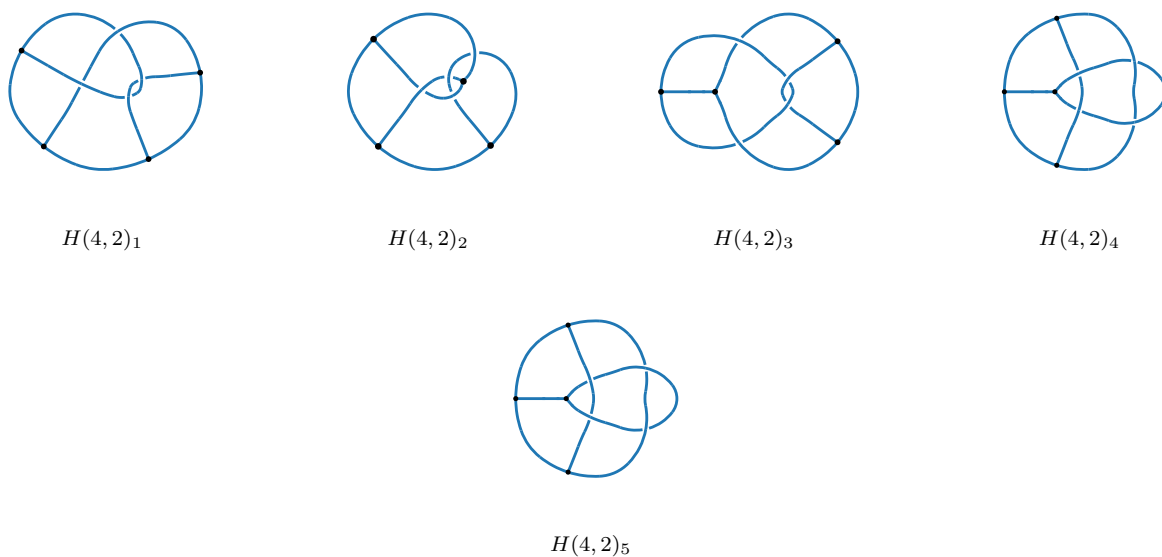


Figure 10: Handcuff-like bonded knots. Each entry is represented by two rows: the diagram and its label.

References

- [1] Vasilina Zayats et al. “Conservation of knotted and slipknotted topology in transmembrane transporters”. In: *Biophysical Journal* 122.23 (2023), pp. 4528–4541.
- [2] Joanna I Sułkowska, Jeffrey K Noel, and Jose N Onuchic. “Energy landscape of knotted protein folding”. In: *Proceedings of the National Academy of Sciences* 109.44 (2012), pp. 17783–17788.

- [3] Joanna I Sulkowska et al. “Conservation of complex knotting and slipknotting patterns in proteins”. In: *Biophysical Journal* 102.3 (2012), 253a.
- [4] Pawel Dabrowski-Tumanski, Andrzej Stasiak, and Joanna I Sulkowska. “In search of functional advantages of knots in proteins”. In: *PloS one* 11.11 (2016), e0165986.
- [5] Shang-Te Danny Hsu. “Folding and functions of knotted proteins”. In: *Current Opinion in Structural Biology* 83 (2023), p. 102709.
- [6] Michal Jamroz et al. “KnotProt: a database of proteins with knots and slipknots”. In: *Nucleic acids research* 43.D1 (2015), pp. D306–D314.
- [7] Mahesh Narayan. “Securing native disulfide bonds in disulfide-coupled protein folding reactions: the role of intrinsic and extrinsic elements vis-a-vis protein aggregation and neurodegeneration”. In: *ACS omega* 6.47 (2021), pp. 31404–31410.
- [8] Boštjan Gabrovšek, Matic Simonič, and Wanda Niemyska. “Classification of (uncolored) bonded knots and links”. In: *arXiv preprint arXiv:2603.18768* (2026).
- [9] Louis H Kauffman. “Invariants of graphs in three-space”. In: *Transactions of the American Mathematical Society* 311.2 (1989), pp. 697–710.
- [10] Colin Adams et al. “Knot theory for proteins: Gauss codes, quandles and bundles”. In: *arXiv preprint arXiv:1912.09353* (2019).
- [11] Boštjan Gabrovšek. “An invariant for colored bonded knots”. In: *Studies in Applied Mathematics* 146.3 (2021), pp. 586–604.
- [12] Gunnar Brinkmann, Brendan McKay, et al. *Fast generation of planar graphs*. University of Kragujevac, 2007.
- [13] Wei Tian et al. “A knot polynomial invariant for analysis of topology of RNA stems and protein disulfide bonds”. In: *Molecular Based Mathematical Biology (Online)* 5.1 (2017), p. 21.